Assignment 2

Name: أصايل حمدان حمود العصيمي

ID: 445000051

Section:2

Machine Learning Course under the supervision of

Dr Afaf Almehmadi

SVM performance Breast Cancer

```
--- Linear SVM ---
Accuracy: 0.9649122807017544
Precision: 1.0
Recall: 0.90625
F1-Score: 0.9508196721311475
ROC-AUC: 0.953125
Confusion Matrix:
 [[107   0]
 [  6  58]]

--- Best RBF SVM ---
Accuracy: 0.9707602339181286
Precision: 0.9836065573770492
Recall: 0.9375
F1-Score: 0.96
ROC-AUC: 0.9640771028037383
Confusion Matrix:
 [[106   1]
 [  4  60]]
```

SVM performance Adult Income

```
SVM Evaluation
Accuracy: Linear=0.813, RBF=0.807
Precision: Linear=0.659, RBF=0.833
Recall: Linear=0.659, RBF=0.366
F1-Score: Linear=0.659, RBF=0.508
ROC-AUC: Linear=0.765, RBF=0.669

Confusion Matrix - Linear SVM
[[95 14]
 [14 27]]

Confusion Matrix - RBF SVM
[[106   3]
 [ 26  15]]
```

Summary:

The performance of SVM on the breast cancer dataset was much better in terms of accuracy, ROC-AUC, and so on. As for the Adult Income dataset, it achieved a higher accuracy result, but it was still the worst when using SVM because the dataset is large and I used a random sample so I could check the results faster.

Clustering performance Breast Cancer

```
Silhouette Score K-Means: 0.3447344346611054
Silhouette Score Hierarchical: 0.6606668813897673
K-Means vs True Labels:
 Diagnosis_num    0    1
row_0
0                13  175
1               344   37
Hierarchical vs True Labels:
 Diagnosis_num    0    1
row_0
1                 0    2
2               357  210
```

Clustering performance Adult Income

```
Silhouette Score K-Means (Adult): 0.12666825546094845
Silhouette Score Hierarchical (Adult): 0.5648107607629006
K-Means vs True Labels (Adult):
 income_num    0    1
row_0
0            53    4
1           309  134
Hierarchical vs True Labels (Adult):
 income_num    0    1
row_0
1             1    1
2           361  137
```

Summary:

For both K-Means and Hierarchical Clustering, the Silhouette Score was used to evaluate cluster quality. The results showed that performance was higher on the Breast Cancer dataset compared to the Adult Income dataset. This is expected because the adult dataset is much larger and more complex, which makes it harder for clustering algorithms to accurately separate the data into meaningful groups.

Key code cells:

Data Preprocessing for Breast Cancer

```
# Load datasets.
data_df=pd.read_csv("/content/wdbc.data", names=columns)
# Show all columns
pd.set_option('display.max_columns', None)
# Show all rows
pd.set_option('display.max_rows', None)
# display the datasets
display(data_df)
```

```
# Print missing values per column
print("Missing values in each column:")
print(data_df.isnull().sum())

# Check if there are any missing values
if data_df.isnull().sum().sum() == 0:
    print("\n No missing values found — Excellent!")
else:
    print("\n There are missing values — you need to handle them.")
print()
```

```
# Encode target variable
print("Target Variable Distribution:")
target_counts = data_df['Diagnosis'].value_counts()
print(target_counts)
data_df['Diagnosis_num'] = np.where(data_df['Diagnosis']=='M', 1, 0)

# Standardize numerical features

features = data_df.loc[:, ~data_df.columns.isin(['ID','Diagnosis','Diagnosis_num'])]
# Select target variable
target = data_df['Diagnosis_num']

# Standardize features to have mean=0 and std=1
scaler = StandardScaler()
features_scaled = scaler.fit_transform(features)
```

```
# split for supervised SVM only (70% training, 30% testing)
X_train, X_test, y_train, y_test = train_test_split(
    features_scaled, target, test_size=0.3, random_state=42, stratify=target
)

print("After preprocessing and splitting:")
print(f"Training samples: {X_train.shape[0]}")
print(f"Testing samples:  {X_test.shape[0]}")

# Train SVM classifier (supervised learning)
clf = svm.SVC()
clf.fit(X_train, y_train)
score = clf.score(X_test, y_test)
print("SVM Accuracy on Test Set:", score)
```

```
# Apply K-Means clustering (unsupervised, no labels used)
kmeans = KMeans(n_clusters=2, random_state=42)
clusters_kmeans = kmeans.fit_predict(features_scaled)
print("K-Means Cluster Counts:", np.bincount(clusters_kmeans))

# Apply Hierarchical Clustering (unsupervised, no labels)
Z = linkage(features_scaled, method='ward')
clusters_hier = fcluster(Z, t=2, criterion='maxclust')
print("Hierarchical Cluster Counts:", np.bincount(clusters_hier))
```

Key code cells:

Data Preprocessing for Adult Income

```
# Load datasets.
adult_df=pd.read_csv("/content/adult.data", names=columns)
# Show all columns
pd.set_option('display.max_columns', 50)
# Show all rows
pd.set_option('display.max_rows', 50)
# display the first 50
adult_df.head(50)
```

```
# Check number of missing values per column
print("Missing values in each column:")
print(adult_df.isnull().sum())

# Number of rows before removing missing values
print("\nNumber of rows before removing missing values:", adult_df.shape[0])

# Drop rows with missing values
adult_df = adult_df.dropna()
```

```
# Fix income values
adult_df['income'] = adult_df['income'].str.replace('.', '', regex=False).str.strip()

# Encode target
adult_df['income_num'] = np.where(adult_df['income'] == '>50K', 1, 0)

# Sample small subset for fast testing (remove stratify since pandas sample() does not support it
sample_df = adult_df.sample(n=500, random_state=42)

# Features and target for sample
X_sample = sample_df.drop(['income','income_num'], axis=1)
y_sample = sample_df['income_num']
```

```
# Encode categorical features for sample
X_sample = pd.get_dummies(X_sample)

# Scale sample features
scaler = StandardScaler()
X_scaled_sample = scaler.fit_transform(X_sample)
```

```
# Train-test split (Supervised models only), stratify here to keep class balance
X_train2, X_test2, y_train2, y_test2 = train_test_split(
    X_scaled_sample, y_sample, test_size=0.3, random_state=42, stratify=y_sample
)

print("Preprocessing done")
print("Train samples:", X_train2.shape[0])
print("Test samples:", X_test2.shape[0])
```

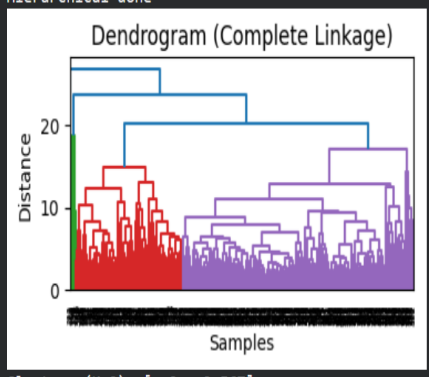
```
# Train SVM classifier (supervised learning)
svm_adult = svm.SVC()
svm_adult.fit(X_train2, y_train2)
print("\nSVM Accuracy:", svm_adult.score(X_test2, y_test2))

# Apply K-Means clustering (unsupervised, no labels used)
kmeans_adult = KMeans(n_clusters=2, random_state=42)
clusters_km_adult = kmeans_adult.fit_predict(X_scaled_sample)
print("K-Means clusters:", np.bincount(clusters_km_adult))

# Apply Hierarchical Clustering (unsupervised, no labels)
Z_adult = linkage(X_scaled_sample, method='ward')
clusters_hier_adult = fcluster(Z_adult, t=2, criterion='maxclust')
print("Hierarchical clusters:", np.bincount(clusters_hier_adult))
```

Main visualizations:

Breast Cancer:

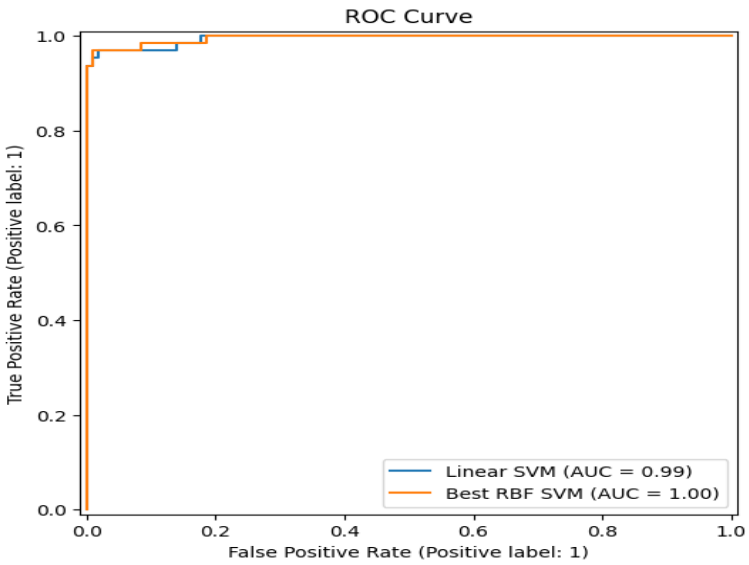
```python
# Step 2: Unsupervised Model Building

# K-Means clustering
kmeans = KMeans(n_clusters=2, random_state=42)
kmeans.fit(features_scaled)
print("K-Means done")

# Hierarchical clustering (single & complete linkage)
Z_single = linkage(features_scaled, method='single')
Z_complete = linkage(features_scaled, method='complete')
print("Hierarchical done")

# Dendrogram for complete linkage
plt.figure(figsize=(4, 2))
dendrogram(Z_complete)
plt.title("Dendrogram (Complete Linkage)")
plt.xlabel("Samples")
plt.ylabel("Distance")
plt.show()

# Cut dendrogram to get 2 clusters
clusters_hier = fcluster(Z_complete, t=2, criterion='maxclust')
print("Clusters (K=2):", np.bincount(clusters_hier))
```

```
K-Means done
Hierarchical done
```



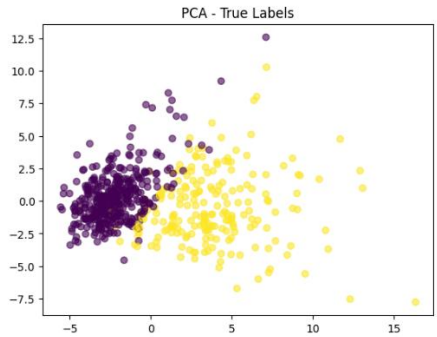

```
Clusters (K=2): [  0   2 567]
```

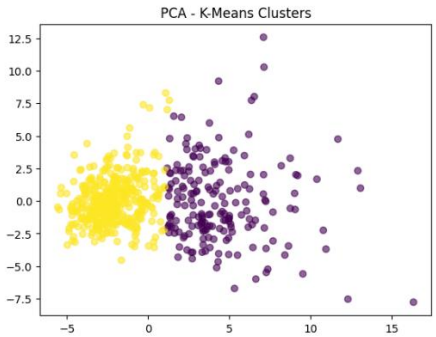
```python
# Step 3: SVM Evaluation and Visualization Supervised
# Predict labels
y_pred_linear = linear_svm.predict(X_test)
y_pred_rbf = best_rbf_svm.predict(X_test)

# Compute metrics and confusion matrix
def print_metrics(name, y_true, y_pred):
    print(f"--- {name} ---")
    print("Accuracy:", accuracy_score(y_true, y_pred))
    print("Precision:", precision_score(y_true, y_pred))
    print("Recall:", recall_score(y_true, y_pred))
    print("F1-Score:", f1_score(y_true, y_pred))
    print("ROC-AUC:", roc_auc_score(y_true, y_pred))
    print("Confusion Matrix:\n", confusion_matrix(y_true, y_pred))
    print()

print_metrics("Linear SVM", y_test, y_pred_linear)
print_metrics("Best RBF SVM", y_test, y_pred_rbf)

# Plot ROC curves
plt.figure(figsize=(8,6))
RocCurveDisplay.from_estimator(linear_svm, X_test, y_test, name="Linear SVM", ax=plt.gca())
RocCurveDisplay.from_estimator(best_rbf_svm, X_test, y_test, name="Best RBF SVM", ax=plt.gca())
plt.title("ROC Curve")
plt.show()
```

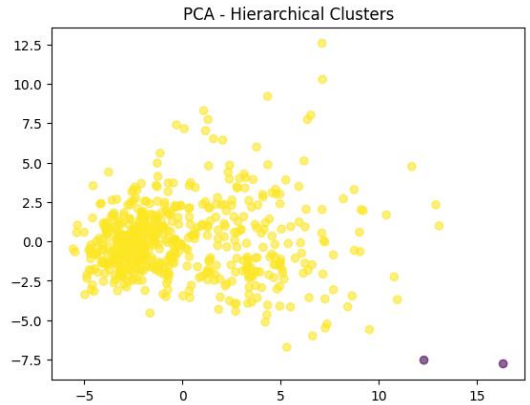


```python
# Step 3: Clustering Evaluation and Visualization Unsupervised
# Silhouette Scores
sil_kmeans = silhouette_score(features_scaled, kmeans.labels_)
sil_hier = silhouette_score(features_scaled, clusters_hier)
print("Silhouette Score K-Means:", sil_kmeans)
print("Silhouette Score Hierarchical:", sil_hier)

# Compare clusters to true labels
kmeans_table = pd.crosstab(kmeans.labels_, target)
hier_table = pd.crosstab(clusters_hier, target)
print("K-Means vs True Labels:\n", kmeans_table)
print("Hierarchical vs True Labels:\n", hier_table)

#  PCA for 2D visualization
pca = PCA(n_components=2)
features_2d = pca.fit_transform(features_scaled)

import matplotlib.pyplot as plt
```

```python
# Plot colored by true labels
plt.scatter(features_2d[:,0], features_2d[:,1], c=target, cmap='viridis', alpha=0.6)
plt.title("PCA - True Labels")
plt.show()

# Plot colored by K-Means clusters
plt.scatter(features_2d[:,0], features_2d[:,1], c=kmeans.labels_, cmap='viridis', alpha=0.6)
plt.title("PCA - K-Means Clusters")
plt.show()

# Plot colored by Hierarchical clusters
plt.scatter(features_2d[:,0], features_2d[:,1], c=clusters_hier, cmap='viridis', alpha=0.6)
plt.title("PCA - Hierarchical Clusters")
plt.show()
```





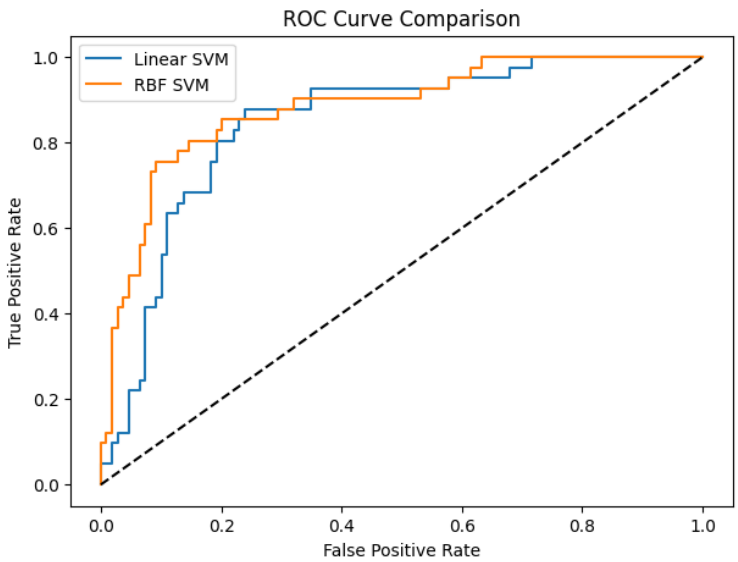


Main visualizations:

Adult Income

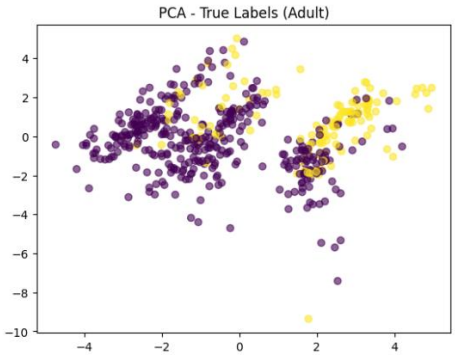
```python
# Step 3: SVM Evaluation and Visualization Supervised

# Predict class labels
y_pred_linear = svm_linear_adult.predict(X_test2)
y_pred_rbf = svm_rbf_adult.predict(X_test2)

# Compute metrics
metrics = {
    "Accuracy": accuracy_score,
    "Precision": precision_score,
    "Recall": recall_score,
    "F1-Score": f1_score,
    "ROC-AUC": roc_auc_score
}

print("SVM Evaluation")
for name, func in metrics.items():
    linear_score = func(y_test2, y_pred_linear)
    rbf_score = func(y_test2, y_pred_rbf)
    print(f"{name}: Linear={linear_score:.3f}, RBF={rbf_score:.3f}")

# Confusion matrices
print("\nConfusion Matrix - Linear SVM")
print(confusion_matrix(y_test2, y_pred_linear))
```

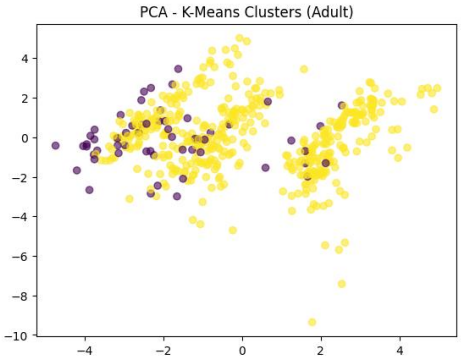
```python
# ROC curves
fpr_linear, tpr_linear, _ = roc_curve(y_test2, svm_linear_adult.decision_function(X_test2))
fpr_rbf, tpr_rbf, _ = roc_curve(y_test2, svm_rbf_adult.decision_function(X_test2))

plt.figure(figsize=(7,5))
plt.plot(fpr_linear, tpr_linear, label="Linear SVM")
plt.plot(fpr_rbf, tpr_rbf, label="RBF SVM")
plt.plot([0,1],[0,1],'k--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison")
plt.legend()
plt.show()
```

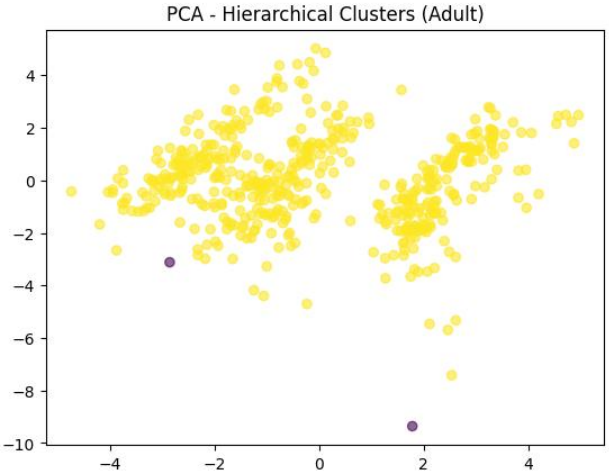


```python
# Step 3: Clustering Evaluation and Visualization Unsupervised

# Silhouette Scores
sil_kmeans_adult = silhouette_score(X_scaled_sample, clusters_km_adult)
sil_hier_adult = silhouette_score(X_scaled_sample, clusters_hier_adult)
print("Silhouette Score K-Means (Adult):", sil_kmeans_adult)
print("Silhouette Score Hierarchical (Adult):", sil_hier_adult)

# Compare clusters to true labels
kmeans_table_adult = pd.crosstab(clusters_km_adult, y_sample)
hier_table_adult = pd.crosstab(clusters_hier_adult, y_sample)
print("K-Means vs True Labels (Adult):\n", kmeans_table_adult)
print("Hierarchical vs True Labels (Adult):\n", hier_table_adult)

# PCA for 2D visualization
pca = PCA(n_components=2)
X_2d = pca.fit_transform(X_scaled_sample)
```

```python
# Plot colored by true labels
plt.scatter(X_2d[:,0], X_2d[:,1], c=y_sample, cmap='viridis', alpha=0.6)
plt.title("PCA - True Labels (Adult)")
plt.show()

# Plot colored by K-Means clusters
plt.scatter(X_2d[:,0], X_2d[:,1], c=clusters_km_adult, cmap='viridis', alpha=0.6)
plt.title("PCA - K-Means Clusters (Adult)")
plt.show()

# Plot colored by Hierarchical clusters
plt.scatter(X_2d[:,0], X_2d[:,1], c=clusters_hier_adult, cmap='viridis', alpha=0.6)
plt.title("PCA - Hierarchical Clusters (Adult)")
plt.show()
```







Comparative Analysis Breast Cancer:

1- How does SVM performance compare to K-Means and Hierarchical?

With accuracy over 96% and robust Precision, Recall, and ROC-AUC scores, the SVM models (both Linear and RBF) performed quite well. This is to be expected as SVM is a supervised learning technique that learns distinct boundaries between benign and malignant instances by using the actual class labels during training.

However, the unsupervised models (K-Means and Hierarchical Clustering) do not know the true labels; instead, they rely solely on feature similarity. Their performance suffered as a result, particularly K-Means, which considerably mixed the two classes. The greater Silhouette Score of Hierarchical Clustering indicates that it performed comparatively better.

2- Which method seems to separate the classes better, and why?

Compared to K-Means and Hierarchical, which only rely on feature similarity, SVM achieves superior accuracy and cleaner class boundaries because it is a supervised method that employs the true labels.

3- How did preprocessing (encoding, scaling) help your models?

The diagnosis labels were transformed into numerical form (0/1) by encoding the target, which is necessary for SVM and quantitative assessment.

All features have a mean of 0 and a standard deviation of 1 thanks to scaling (standardization), which prevents features with wide numerical ranges from taking precedence over those with narrow ranges. This facilitates the fair treatment of all features by both SVM and clustering.

Proper preprocessing allowed SVM to achieve stable classification and good accuracy. Although clustering is still less effective than supervised learning, clustering (K-Means/Hierarchical) also had a clearer feature space to group.

4- which method (SVM, K-Means, Hierarchical) would you recommend, and why?

I would recommend SVM,

because it is a supervised method that learns the true class boundaries. It achieved much higher accuracy and stronger evaluation metrics compared to K-Means and Hierarchical Clustering, which struggled since they do not use the real labels.

Comparative Analysis Adult Income:

1- How does SVM performance compare to K-Means and Hierarchical?

On the Adult dataset, the SVM models performed poorly, with weak recall and moderate accuracy, particularly for the RBF model. The clustering techniques, on the other hand, also had difficulties; nonetheless, Hierarchical Clustering outperformed K-Means in terms of Silhouette Score, suggesting a more distinct division between groups. The complexity and imbalance of the adult dataset had a negative impact on all techniques, while SVM performed marginally better overall in identifying income classes.

2- Which method seems to separate the classes better, and why?

The SVM method separates the classes better because it is supervised and learns directly from the true class labels, while K-Means and Hierarchical clustering are unsupervised and do not know the actual classes, so their separation is less accurate.

3- How did preprocessing (encoding, scaling) help your models?

Encoding categorical features

transformed text categories into numerical values for processing and understanding by models such as SVM and K-Means. made it possible for the algorithms to efficiently utilize every feature.

Scaling numerical features

To avoid large-scale features (like hours-per-week) overpowering smaller ones (like age), place all features on the same range. increased the accuracy of K-Means distance computations and significantly enhanced SVM performance.

4- which method (SVM, K-Means, Hierarchical) would you recommend, and why?

I recommend SVM

because it handles large, complex datasets better and gives higher accuracy by learning directly from the true labels, while K-Means and Hierarchical often fail to match the real classes.

The Reviewer

ChatGPT help me with the codes